MentorDesk

PRODUCT REQUIREMENTS DOCUMENT

Intern Question & Mentor Response Portal

Document Control

Document field	Value
Version	1.0 — Planning draft
Organization	TriFin-Loop R&D
Project / owner	MentorDesk / Mahmuda Khatun
Status	Proposed for supervisor review; not approved for development
Frontend	React.js (proposed)
Backend	Node.js + Express.js (proposed; approval pending)
Database	PostgreSQL (proposed; approval pending)
Scope	A web application for interns to submit questions and mentors to respond and track status.
Document status	Planning document only; technical choices and open decisions require supervisor review.

This document describes a proposed MVP. It does not claim that the application has been implemented.

Executive Summary

MentorDesk is a proposed responsive web application that centralizes intern questions and mentor responses. Interns can submit questions, view their own question history, follow status changes, and add follow-up comments. Mentors can review submitted questions, provide responses, and update question status. The planned MVP emphasizes a clear question-to-response workflow, role-based access, and visible progress.

This is a planning specification for a 20-day internship assignment. The application is not represented as a working system. The proposed stack is React.js, Node.js/Express.js, and PostgreSQL, subject to supervisor approval.

How to Use This PRD

Requirements and design choices are proposed assumptions until reviewed. The supervisor should confirm roles, workflow, status names, stack, and MVP boundaries before implementation.

Version History

Version	Change
0.1	Initial problem, scope and requirements outline
1.0	Expanded planning PRD, diagrams, backlog, risks, API and test plans

01 | Problem, Users & Product Goals

Problem Statement

Intern questions and mentor replies may be spread across informal messages, email, or separate documents. This can make it difficult for interns to track progress and for mentors to review unresolved questions consistently. MentorDesk proposes a single place to submit questions, respond, update status, and review question history.

Target Users

Persona	Needs and primary actions
Intern	Submit a question with a title, description, category and priority; view own questions; follow status; add follow-up comments.
Mentor	View submitted questions; review details; add responses; update status; follow unresolved items.
Administrator (proposed)	Potentially manage users or roles and view operational summaries; exact responsibilities require supervisor approval.

Product Goals & Proposed Success Measures

- A valid intern question can be submitted once and appears in the intern's own question list.
- A mentor can review a question, add a response, and move it through the agreed status lifecycle.
- An intern cannot read or modify another intern's private questions by changing an identifier.
- Users can identify open and unresolved work from the dashboard or filtered question list.
- The primary workflow can be demonstrated using synthetic test accounts and sample questions.

Assumptions to Validate

- The project is a demonstration MVP and will use synthetic data only.
- Intern and Mentor are the primary roles; Administrator is proposed and needs confirmation.
- The proposed technology stack will be confirmed by the supervisor before development.
- Email notifications, file uploads, and production deployment are not assumed to be part of the initial MVP.

02 | Scope & MVP Feature Inventory

In Scope — Must Have

ID	Capability
F01	Register, sign in and sign out using role-aware accounts
F02	Create a question with title, category, description and priority
F03	View own questions and question details
F04	Mentor views submitted questions and reviews a question
F05	Mentor adds a response and updates question status
F06	Track status: Open, In Review, Answered, Resolved
F07	Add follow-up comments to a question
F08	Search/filter questions by status and relevant fields
F09	Dashboard summaries and unresolved-question view

F10	Server-side role and ownership checks

Should Have / Later

- Pagination for large question lists.
- Additional dashboard metrics and configurable categories.
- Email notifications after a response or status change, if approved.
- Accessibility and responsive refinements beyond the primary workflow.

Out of Scope for Initial Planning MVP

- AI-generated answers or automatic question classification.
- Live chat or video calls.
- Native mobile applications.
- Production SSO or integration with external identity providers.
- File uploads until storage, validation and access controls are designed.

Acceptance Boundary

The planned MVP is demonstrable when an intern can submit and track a question, a mentor can respond and update its status, and access restrictions prevent users from accessing records they do not own. A landing page alone does not meet this boundary.

03 | User Stories & Process Flow

Core User Stories

- US-01: As an intern, I want to submit a question so that a mentor can review it.
- US-02: As an intern, I want to view my question history and current status so that I know whether my question is being handled.
- US-03: As a mentor, I want to review submitted questions and filter unresolved work so that I can prioritize responses.
- US-04: As a mentor, I want to add a response and update status so that the intern can see progress.
- US-05: As an intern, I want to add a follow-up comment so that I can clarify a question or respond to a mentor.

Example Journey — Intern

1. The intern opens MentorDesk and signs in.
2. The intern opens the dashboard and selects Submit Question.
3. The intern enters title, category, description and priority.
4. The system validates required fields and saves the question with Open status.
5. The question appears in My Questions with a unique question identifier.
6. The intern opens the question detail page to review status, responses and follow-up comments.

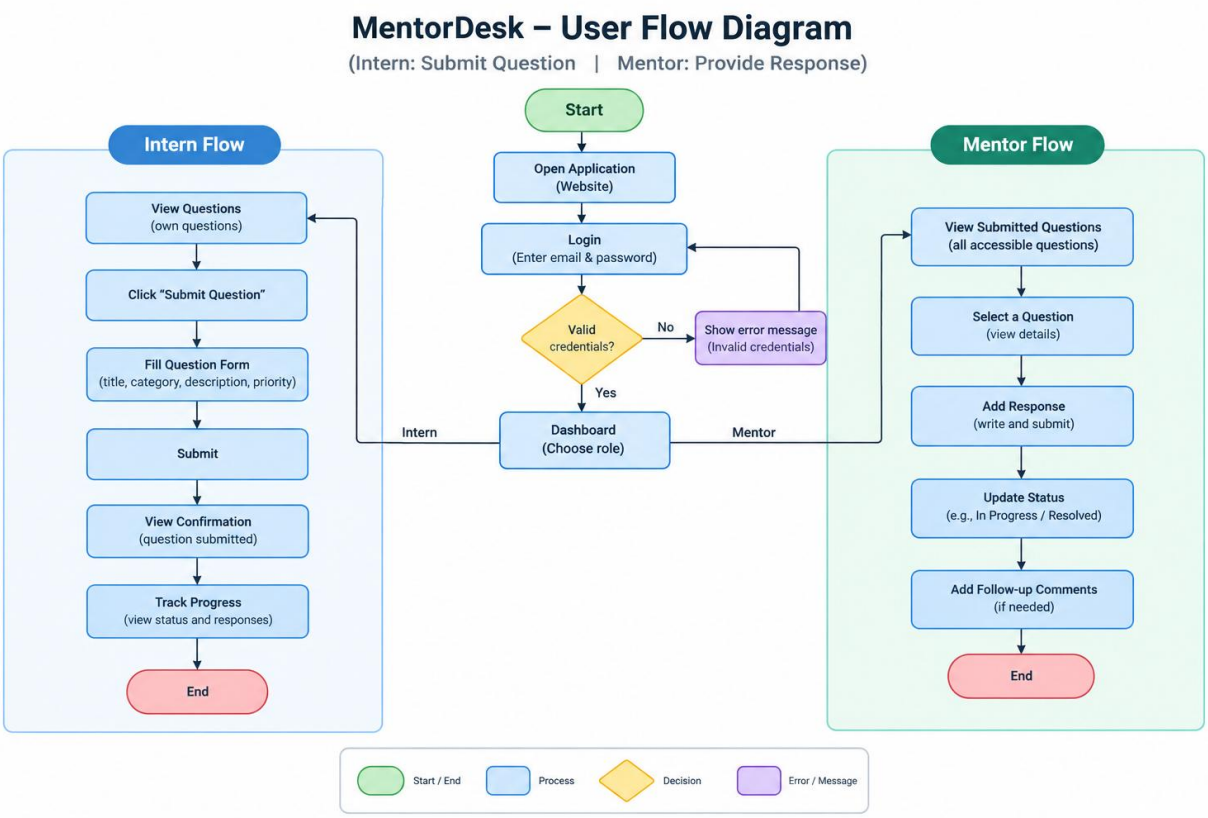
Example Journey — Mentor

7. The mentor signs in and opens the submitted-question list.
8. The mentor filters or selects a question and reviews its details.
9. The mentor adds a response and updates status to In Review or Answered as appropriate.
10. The intern can view the response and current status; follow-up comments may continue the conversation.
11. When the issue is complete, the question may be marked Resolved.

Proposed Lifecycle & Exceptions

Proposed status values: Open, In Review, Answered, and Resolved. The supervisor should confirm whether a Resolved question can be reopened. Invalid forms should remain unsaved and show clear field-level errors.

User Flow Diagram



04 | Functional Requirements

Requirement	Actor	Behavior	Acceptance criteria
FR-01 — Authentication	Intern/Mentor	User signs in or out using valid credentials; session state is handled safely.	Valid credentials open the appropriate dashboard; invalid credentials show a generic error.
FR-02 — Submit question	Intern	Submit title, category, description and priority.	A valid submission creates one question, defaults to Open, and appears in My Questions.
FR-03 — View own questions	Intern	List and open the intern's own questions.	Only questions owned by the signed-in intern are returned.
FR-04 — Review questions	Mentor	View and open submitted questions.	Mentor sees the question list and details permitted by role.
FR-05 — Respond to question	Mentor	Add a response to a question.	Response is linked to the question and shows author and timestamp.
FR-06 — Update status	Mentor	Change status to an allowed lifecycle value.	Invalid transitions are rejected; status changes are recorded.
FR-07 — Follow-up comments	Intern/Mentor	Add a comment to a question thread.	Comment is linked to the question and author and displayed in order.
FR-08 — Search/filter	Intern/Mentor	Filter by status and search relevant question text.	Results respect role and ownership rules; empty states are helpful.
FR-09 — Dashboard summaries	Intern/Mentor	View counts and unresolved items relevant to the role.	Counts reflect the available authorized records.
FR-10 — Authorization	All roles	Access protected pages and API endpoints.	Server verifies identity, role and record ownership for every protected operation.

Validation Rules (Proposed)

- Title is required and should be limited to a reasonable length (proposed: 5–100 characters).
- Description is required and should be limited to a reasonable length (proposed: 20–2,000 characters).
- Category and priority must be selected from allowed values.
- All inputs are validated on the server, even when the frontend also validates them.
- Responses, comments and status changes must be associated with an authenticated user and a valid question.

05 | Non-Functional Requirements

ID	Requirement	Verification approach
NFR-01 Security	Hash passwords; enforce authorization on the server; validate input; keep secrets out of source control.	Attempt invalid login, role tampering and unauthorized record access.
NFR-02 Performance	Proposed target: common list views respond within 2 seconds on a local demo dataset of up to 1,000 questions.	Measure browser/network timing with seeded test data.
NFR-03 Usability	Primary workflow should work on mobile and desktop; forms have labels, visible focus and clear error messages.	Review keyboard navigation and viewport widths.
NFR-04 Data integrity	Question, response and comment records maintain valid relationships; failed operations should not leave partial records.	Test validation failures and forced database errors.
NFR-05 Maintainability	Separate frontend, API routes/business logic and data access; document setup in README.	Review repository structure and setup instructions.
NFR-06 Reliability	Show useful error states when a request fails; do not display false success messages.	Test unavailable API and failed save operations.
NFR-07 Compatibility	Support current mainstream desktop browsers and responsive mobile layouts.	Manual cross-browser and responsive checks.
NFR-08 Privacy	Use synthetic demo data only; restrict access	Review sample data and perform ownership

	to authorized records.	tests.

Dependencies & Constraints

Local development is proposed to require Node.js, npm, Git/GitHub, and PostgreSQL. Hosting is optional and requires supervisor approval. Do not place real private user information, credentials, or confidential questions in the demonstration project.

es marked proposed must be reviewed with the supervisor.